Лабораторна робота №11

Тема: React Router 8, форми, валідація та автентифікація.

Мета: навчитися будувати багатосторінкову SPA-структуру на основі React Router, працювати з route params і query params, створювати повторно використовувану форму в діалоговому вікні, організовувати authentication та authorization для дій із даними.

Лабораторна є продовженням **Study Planner**. Потрібно зберегти попередній REST/API-функціонал і додати навігацію та автентифікацію.

Вимоги до звіту

1. Використовувати React 19.x (не нижче 19.2.7) та Node.js 22.22 або новішу версію.
2. Використовувати TypeScript.
3. Використати актуальний React Router 8.
4. Реалізувати динамічний маршрут із route params.
5. Використати query params для фільтрації або сортування.
6. Реалізувати спільний layout.
7. Реалізувати форму Login з React Hook Form.
8. Реалізувати схему валідації за допомогою Zod.
9. Реалізувати Auth Context.
10. Реалізувати login, logout та protected route.
11. Реалізувати спільний `TaskFormDialog` для створення та редагування завдання.
12. Створення, редагування та видалення завдань повинні бути доступні лише авторизованому користувачу.
13. Під час авторизованих API-запитів передавати token через `Authorization: Bearer <token>`. Для рекомендованого DummyJSON це обов'язково для `/auth/me`.
14. Не використовувати `alert`.
15. Для стилізації використати один із підходів: SCSS із дотриманням BEM, Bootstrap або Tailwind CSS.
16. Якщо обрано SCSS, дотримуватися BEM-патерну для класів.
17. Дотримуватися ESLint, TypeScript та Conventional Commits.

Теоретична частина

1. DummyJSON Auth.
2. DummyJSON Users.

Опрацювати:

1. React Router 8.
2. Routing.
3. Navigating.
4. URL values.
5. React Hook Form.
6. Zod.
7. Матеріали лекцій щодо Routing, Forms, Validation та Authentication.
8. Поняття authentication, authorization, token та protected route.

Практична частина

Версії та встановлення React Router

Для цієї лабораторної використовується **React Router 8**. У React Router 8 пакет `react-router-dom` більше не використовується: основні API імпортуються з `react-router`, а DOM-специфічний `RouterProvider` — з `react-router/dom`.

Встановити:

```
npm install react-router
```

Для базової частини лабораторної використовувати Declarative Mode з `BrowserRouter`, `Routes` та `Route`. Data Router з `createBrowserRouter`, `loader` та `redirect` можна використати як додатковий варіант.

Приклад базового імпорту:

```
import {
  BrowserRouter,
  Routes,
  Route,
  Link,
} from 'react-router';
```

Не використовувати приклади з `react-router-dom`, розраховані на старі версії React Router.

Стилізація та BEM

Для стилізації обрати один із трьох підходів:

1. **SCSS + BEM** — власні стилі з класами за схемою Block__Element—Modifier;

2. **Bootstrap** — готові utility та component класи;
3. **Tailwind CSS** — utility-first класи.

Не потрібно поєднувати всі три підходи одночасно.

Якщо обрано **SCSS + BEM**:

- стилі організувати за компонентами або логічними блоками;
- кожен компонент або логічний UI-блок має власний `.scss` файл поруч із відповідним компонентом;
- не зберігати стилі компонентів у загальному `App.css` або іншому великому спільному файлі;
- назви класів повинні відповідати BEM;
- не прив'язувати стилі до глибокої структури DOM;
- глобальні стилі залишати лише для справді глобальних речей: `body`, базові стилі, шрифти, reset тощо.

Git workflow

Рекомендується виконувати роботу у feature-гілці `feature/routing-auth`.

Приклад Conventional Commit:

```
feat(lab-11): add routing and protected auth
```

1. Структура маршрутів

Розширити Study Planner такими маршрутами:

```
/
 ├──── /tasks
 ├──── /tasks/:taskId
 ├──── /login
 └──── /profile
```

Маршрут `/` може перенаправляти на `/tasks`.

2. Layout та навігація

Створити спільний layout застосунку з:

- Header;
- навігацією;

- основною областю контенту.

Навігація між сторінками повинна працювати без повного перезавантаження браузера.

У навігації передбачити посилання на:

- Tasks;
- Profile;
- Login/Logout відповідно до стану сесії.

3. Route Params та сторінка деталей

На сторінці деталей отримувати `taskId` з URL.

Приклад:

```
/tasks/42
```

Для отримання даних використати repository з лабораторної №10.

На сторінці показати щонайменше:

- назву;
- опис;
- категорію;
- пріоритет;
- стан виконання;
- хто створив завдання;
- дату створення.

Поля `createdBy` та `createdAt` повинні належати domain-моделі `Task`. Якщо обраний API не надає ці поля, вони можуть бути client-owned metadata. У такому разі потрібно явно позначити їх у коді як дані, якими володіє клієнт, а не сервер.

4. Query Params

Фільтр Study Planner перенести у URL.

Приклад:

```
/tasks?status=completed&priority=high
```

Після оновлення сторінки параметри повинні зберігатися.

Один і той самий фільтр не потрібно одночасно дублювати в незалежному state та URL без чіткої причини.

5. TaskFormDialog — створення та редагування

Створити один повторно використовуваний компонент `TaskFormDialog`, який використовується для двох сценаріїв:

- **create** — відкривається кнопкою `+` на сторінці `/tasks`;
- **edit** — відкривається кнопкою «Редагувати» на сторінці `/tasks/:taskId`.

Не потрібно створювати окремі `CreateTaskForm` та `EditTaskForm`. Одна форма повинна працювати в двох режимах, отримуючи через props режим та початкові дані.

Приблизний API компонента:

```
type TaskFormDialogProps = {
  open: boolean;
  mode: "create" | "edit";
  task?: Task;
  onClose: () => void;
  onSubmit: (data: TaskFormData) => Promise<void>;
};
```

Форма повинна містити:

- назву;
- опис;
- категорію;
- пріоритет.

Використати React Hook Form та Zod. Під час відправлення показувати pending state та блокувати кнопку Submit.

Після успішного create/edit:

- закрити dialog;
- оновити дані в UI без повного перезавантаження сторінки;
- показати короткий success state.

Dialog повинен коректно закриватися через кнопку Close/Cancel. Обов'язково передбачити базову доступність: зрозумілий заголовок, keyboard focus та закриття через Escape.

6. Authorization для CRUD-дій

Перегляд списку та деталей завдання можна залишити доступним без авторизації.

Для неавторизованого користувача такі дії повинні бути недоступними:

- створення нового завдання;
- редагування;
- видалення.

Приклад сценарію:

```
Гість → /tasks → /tasks/42 → «Редагувати» → Login
Авторизований → /tasks → «+» → TaskFormDialog → create
```

Не потрібно покладатися лише на приховування кнопок. Перед виконанням захищеної дії потрібно перевіряти стан авторизації.

Для редагування та видалення після перевірки auth виконати відповідну repository-операцію з Authorization header. === 7. Форма Login

Створити сторінку `Login`.

Для рекомендованого DummyJSON використовувати поля:

- username;
- password.

Використати React Hook Form та Zod.

Мінімальна схема:

- username — обов’язковий;
- password — обов’язковий, щонайменше 6 символів у межах цієї лабораторної.

Помилки повинні показуватися біля відповідних полів.

Після коректної відправки форми виконати login-flow.

8. Auth Context та token

Створити `AuthContext`, який зберігатиме щонайменше:

- поточного користувача;
- `accessToken` або ознаку активної сесії;
- `isAuthenticated`;
- `login`;
- `logout`.

Для рекомендованого DummyJSON використовувати реальний authentication flow:

```
POST https://dummyjson.com/auth/login
```

Тіло запиту:

```
{
  "username": "emilys",
  "password": "emilyspass"
}
```

DummyJSON повертає `accessToken` і `refreshToken`.

Для отримання поточного користувача:

```
GET https://dummyjson.com/auth/me
Authorization: Bearer <accessToken>
```

Для базової частини достатньо login, logout, Auth Context та protected route. Refresh token flow належить до додаткових завдань.

Authentication відповідає на питання «хто користувач?», а authorization — «які дії йому дозволені?». У цій лабораторній authorization демонструється через create/edit/delete для завдань.

Вимога щодо мінімальної довжини пароля тут є лише навчальною перевіркою форми, а не універсальним правилом безпеки.

Для альтернативного API студент адаптує API client/repository до його контракту.

API, що використовується в роботі

Рекомендований DummyJSON надає:

Метод	Endpoint	Призначення
POST	`/auth/login`	логін та отримання `accessToken` і `refreshToken`;
GET	`/auth/me`	отримання поточного авторизованого користувача;
POST	`/auth/refresh`	оновлення токенів.

Тестові облікові дані для базової роботи: `emilys` / `emilyspass`. Не потрібно зберігати реальні персональні дані або секрети в репозиторії. === 9. Protected Route

Маршрут `/profile` зробити доступним лише авторизованому користувачу.

Неавторизований користувач повинен перенаправлятися на `/login`.

Після успішного входу користувач повинен отримати доступ до protected route.

10. Авторизовані API-запити

Для запитів, що потребують автентифікації, передавати token через заголовок:

```
Authorization: Bearer <token>
```

Ця логіка повинна бути зосереджена в API client або іншому спільному модулі, а не дублюватися в кожному компоненті.

11. Profile та Logout

На `/profile` показати:

- username або email користувача;
- статус авторизації;
- кнопку Logout.

Після Logout protected routes повинні знову стати недоступними.

12. Додаткові завдання

1. Повернення на попередній маршрут після успішного входу.
2. Додати реєстрацію.
3. Реалізувати окремий `NotFound` для 404.
4. Додати nested routes.
5. Додати роль `admin` та адміністративний маршрут.
6. Зробити lazy loading для однієї сторінки.
7. Відновлювати сесію після перезавантаження.
8. Реалізувати refresh token flow.

Рекомендована структура

```
src/
 ├──── api/
 ├──── repositories/
 ├──── components/
 │     └──── TaskFormDialog.tsx
 ├──── context/
 │     └──── AuthContext.tsx
 ├──── hooks/
 ├──── layouts/
 │     └──── AppLayout.tsx
```

```
├──── pages/
│     ├──── TasksPage.tsx
│     ├──── TaskDetailsPage.tsx
│     ├──── LoginPage.tsx
│     ├──── ProfilePage.tsx
│     └──── NotFoundPage.tsx
├──── schemas/
│     ├──── authSchema.ts
│     └──── taskSchema.ts
├──── routes/
└──── App.tsx
```

Структура може бути іншою, але routing, auth та page-level code повинні мати зрозуміле розділення відповідальності.

Демонстрація на захисті

За час захисту необхідно продемонструвати:

1. список `/tasks` та відкриття `/tasks/:taskId`;
2. відкриття `TaskFormDialog` через `+` для створення;
3. відкриття того самого dialog для редагування з Task Details;
4. Login з помилкою Zod-валідації та успішним входом;
5. спробу захищеної дії без авторизації та перехід на Login;
6. виконання create/edit/delete після авторизації;
7. Logout і повторну перевірку захищеної дії.

У коді достатньо коротко показати `AuthContext`, `TaskFormDialog` та місце, де формується Authorization header.

Не потрібно демонструвати всі додаткові можливості.

Контрольні запитання

1. Що таке SPA і як React Router допомагає реалізувати навігацію?
2. Чим route params відрізняються від query params?
3. Для чого URL можна розглядати як частину стану застосунку?
4. Що таке nested routes і layout?
5. Як працює protected route?
6. Що таке authentication і чим вона відрізняється від authorization?
7. Що таке token?
8. Чому Authorization header зручніше централізувати в API client?

9. Що має містити Auth Context?
10. Чому автентифікацію не варто реалізовувати окремо в кожному компоненті?
11. Що таке schema validation і для чого використовується Zod?
12. Як React Hook Form взаємодіє із Zod?
13. Які ризики пов’язані зі зберіганням token у localStorage?
14. Для чого потрібен redirect на Login у protected route?